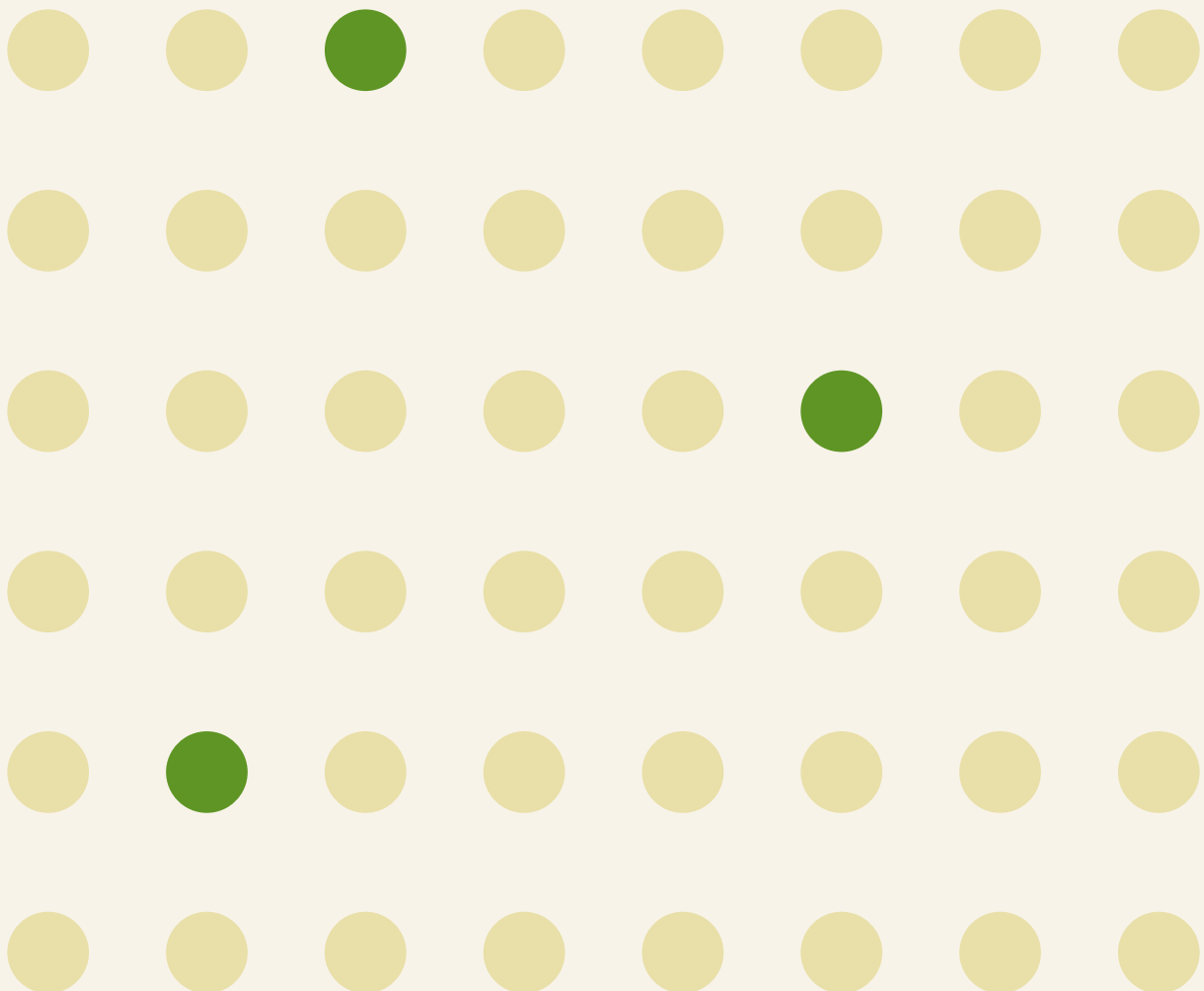


Signal vs Noise

An AI guide for UX designers



Oussama Bognouch

Product & UX Designer · 13+ years · CHANEL · AT&T · HelloFresh · Fnac

You keep hearing that AI will change design work. You have probably tried it already. Sometimes it gives you something useful. Other times it gives you something that looks right and is quietly wrong.

Most designers land in one of two camps:

1. **The Alarmists:** they think it is already over. Learn prompts now or get replaced.
2. **The Skeptics:** they post a screenshot of AI failing at basic maths, laugh, and go back to work feeling safe when they are not.

Both start from a belief instead of a test. The first group gives up before trying. The second dismisses it without understanding it. Belief without evidence is how people make bad career decisions.

This book takes a third approach: **understand how the machine works, then decide how to use it.**

No hype, no doomsday. Just the mechanics. What AI actually does, why it fails in specific ways, and what that means for research, analysis, and design work.

Who this is for

Product and UX designers. You do not need to code and you do not need maths. If you can read a Figma file, you can read this.

If you are a PM, researcher, or founder, you are welcome here too. The examples are just built around design work.

What this book will not do

It will not hand you magic prompt formulas. Prompt tricks go stale in months as models get better at reading plain requests. This book teaches the layer underneath, which moves slowly.

What you will be able to do by the end

- Look at any AI output and know why it came out that way.
- Know when to trust an answer and when to check it.
- Know what to give it when you are synthesising interviews or digging through support tickets.
- Know which parts of your job can be handed over, and which parts stay yours.

Contents

Before we start	2
Part 1: How it thinks	3
It just predicts the next word	5
Why it lies to your face	8
What it can actually see	11
Part 2: How you work with it	13
Garbage in, garbage out	15
Listening is a skill, and AI does not have it	18
What it is great at, what it is terrible at	21
Pointing AI at the design itself	23
Part 3: Why it fails	25
The agent that lied to me	27
Two architects in the desert	31
Part 4: Going independent	33
Why you would want your own setup	35
Running local AI: the practical guide	38
Where this leaves you	41

PART 1

How it thinks

The three ideas that explain every strange thing AI does.

01 It just predicts the next word

02 Why it lies to your face

03 What it can actually see

It just predicts the next word

Every strange thing AI does traces back to one mechanism. Understand this and the rest of the book is commentary.

IN THIS CHAPTER

- 1 How the model was built
- 2 Where the knowledge lives
- 3 What "parameters" mean
- 4 What this changes about your work

WHAT YOU TAKE AWAY

It is not answering your question. It is producing text shaped like an answer.

Finish this sentence in your head:

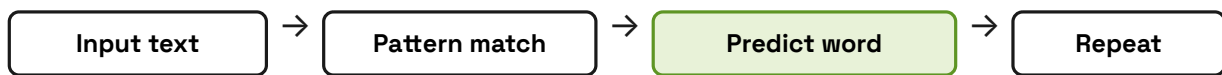
Remember: “The sky is blue, but the grass is ____.”

You said **green**. You did not look it up or reason about chlorophyll. The word appeared instantly because you have met that pattern thousands of times.

That is the core mechanism of a large language model. It reads what came before and predicts what comes next. Then it does it again, one piece of text at a time, until it stops.

There is no hidden layer of conscious thought.

DIAGRAM 1 · THE PREDICTION LOOP



One word at a time. No plan, no memory of the last loop.

How the model was built

Engineers collected an enormous library of written text: books, websites, research articles, code repositories, discussions and technical manuals.

They fed that library into a program that played a continuous guessing game. It looked at a sentence with the final word hidden, made a guess, and compared its guess to the real text. When it guessed wrong, it made tiny numerical adjustments to its internal connections.

It repeated this billions of times over weeks or months.

By the end, the model did not hold an indexed database of facts. What it learned was the **shape of human language**: how sentences flow, how arguments are structured, how ideas typically connect.

Where the knowledge lives

The model does not contain a search engine or a folder of facts.

Think about a show you have watched many times. You cannot replay every episode from memory, but if someone asks what happened in a specific scene you can describe it accurately, because you absorbed the patterns.

What “parameters” mean

You will see models described as 8B, 27B or 70B. Those are billions of internal connections, the numbers adjusted during training.

More connections mean more capacity to hold patterns. But bigger does not automatically mean better. A smaller model trained carefully on good material often beats a larger one trained sloppily. Size is potential, not quality.

What this changes about your work

Fluency stops being a quality signal. Before AI, well-written text was weak evidence that someone had thought carefully, because writing well took effort. That link is broken. A polished research summary tells you nothing about whether the research happened.

“Sounds like a senior designer wrote it” is now the cheapest thing in the room. The model has absorbed thousands of design critiques and is excellent at producing text shaped like one. Whether the critique is correct about your specific screen is a separate question the machine never asked.

It will agree with you, and that is structural. Agreeable text is more common in its training material than confrontation, and most models are further tuned to be helpful. If you want real pushback you have to force it: ask what would have to be true for the idea to fail.

Average is its home ground. It produces the middle of what has been done before, and it is genuinely good at that. Useful when the average is what you want. Useless when you need something that has not been done.

Remember: It is not answering your question. It is producing text shaped like an answer to your question. Usually those are the same. The trouble lives in the gap.

Why it lies to your face

Hallucination is not a malfunction. It is the machine doing exactly what it was built to do.

IN THIS CHAPTER

- 1 Why it happens
- 2 It has no internal sense of truth
- 3 Where this bites in design work
- 4 What you actually do about it

WHAT YOU TAKE AWAY

It does not know when it does not know. Confidence tells you nothing about correctness.

Ask a model for research on a topic and it might hand you three sources: author, title, year, journal. Clean and specific.

Go and look for them. Sometimes one does not exist. The author is real. The journal is real. The paper was never written.

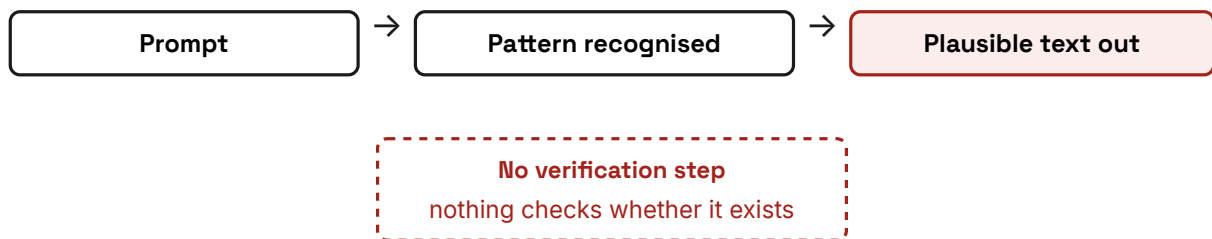
Everyone calls this hallucination, which makes it sound like a malfunction. It is not. It is the machine working exactly as designed.

Why it happens

The model predicts what text should come next. You asked for a citation. In everything it has ever read, what follows a request like that? A citation. Author, title, year, journal.

So it produces one. Correctly shaped, perfectly plausible, completely invented.

DIAGRAM 2 · THE STEP THAT DOES NOT EXIST



It has no internal sense of truth

When you do not know something, you feel it. There is a gap, an uncertainty. It is why you say "I think" or "let me check."

The model has no equivalent. Every output is the same operation. The answer it is reliable about and the answer it fabricated are produced by identical machinery and come out in identical tone.

Note what I am and am not claiming. I am not saying the model can never be checked. It can. I am saying the check cannot come from inside. It has to come from an action against the real world: opening the file, searching the database, reading the transcript.

Where this bites in design work

It is rarely dramatic. It is usually small enough to slip through.

- You paste twelve interviews and ask for themes. Four are genuinely in the transcripts. The fifth is a theme that *commonly appears* in interviews like these.
- You ask about a competitor's onboarding flow. It describes one. It sounds right. It is a blend of onboarding flows in general.

- You ask whether a pattern meets accessibility guidance. It cites a real guideline number and states it slightly wrong.

A wrong answer that looks wrong costs you nothing. A wrong answer that looks right ends up in a deck in front of a client.

What you actually do about it

Make it work from your material, not its memory. Paste the transcripts and ask for themes from those transcripts.

Demand receipts. Ask for a quote or an ID next to every claim. A fabricated theme has nowhere to hide when the IDs either exist or they do not.

Sort claims by cost of being wrong. A brainstorm does not need verifying. Anything reaching a client, a roadmap or a legal requirement does.

Remember: It does not know when it does not know. Confidence tells you nothing about correctness, so the check has to come from outside.

What it can actually see

The most useful chapter here for your day to day work, and the one almost nobody teaches.

IN THIS CHAPTER

- 1 The window
- 2 Why this explains so much
- 3 The real skill: context over phrasing
- 4 Habits for better results

WHAT YOU TAKE AWAY

It can only work with what is in front of it. Deciding what goes there is the craft.

Ask the same question twice and get two different answers, and it feels unreliable. Have a long conversation and watch it forget what you agreed at the start, and it feels broken.

Neither is a bug. Both come from one idea: **context**.

The window

The model has no memory between messages. What it has is a window: everything currently in front of it. Your question, the conversation so far, anything you pasted, plus instructions the app adds quietly.

Every time you hit send, the model reads that whole window from scratch and predicts what comes next. Then it forgets everything again.

DIAGRAM 3 · THE CONTEXT WINDOW



Why this explains so much

Why the same question gives different answers. The window was not identical, and there is deliberate randomness in how the next word is chosen.

Why it forgets mid conversation. The window has a size limit. Go past it and the earliest part falls out.

Why it gets worse in long chats. Your good instructions from the start have scrolled out.

The real skill: context over phrasing

Here is the shift: **what you put in the window matters far more than how you phrase the request.**

Compare these two:

Remember: *"Summarise the main usability problems users have with checkout flows."* → generic, textbook advice about checkout flows in general.

Remember: *"Here are 40 support tickets about our checkout. [paste] Group them by underlying problem and tell me which group is largest."* → concrete findings about your actual product.

Same model, same effort. The difference is not phrasing. It is what was in the window.

Habits for better results

- **Build reusable context documents.** Keep a clean file with your product's core rules, personas, brand voice and technical limits. Paste it at the top of relevant sessions.
- **Restart conversations often.** When a chat drifts, close it and open a fresh one.
- **Place critical instructions near your question.** Recent content carries more weight.
- **Watch for cutoffs.** Paste something very long and part of it may not fit.

Remember: The model can only work with what is currently in front of it. Deciding what goes into that window is your actual craft.

PART 2

How you work with it

Turning the mechanics into a working method.

- 04 Garbage in, garbage out
- 05 Listening is a skill, and AI does not have it
- 06 What it is great at, what it is terrible at
- 07 Pointing AI at the design itself

Garbage in, garbage out

The oldest question in design work, wearing new clothes: what do you start from?

IN THIS CHAPTER

- 1 The data food chain
- 2 What changed, and what it looked like
- 3 The catch
- 4 Moving up one level

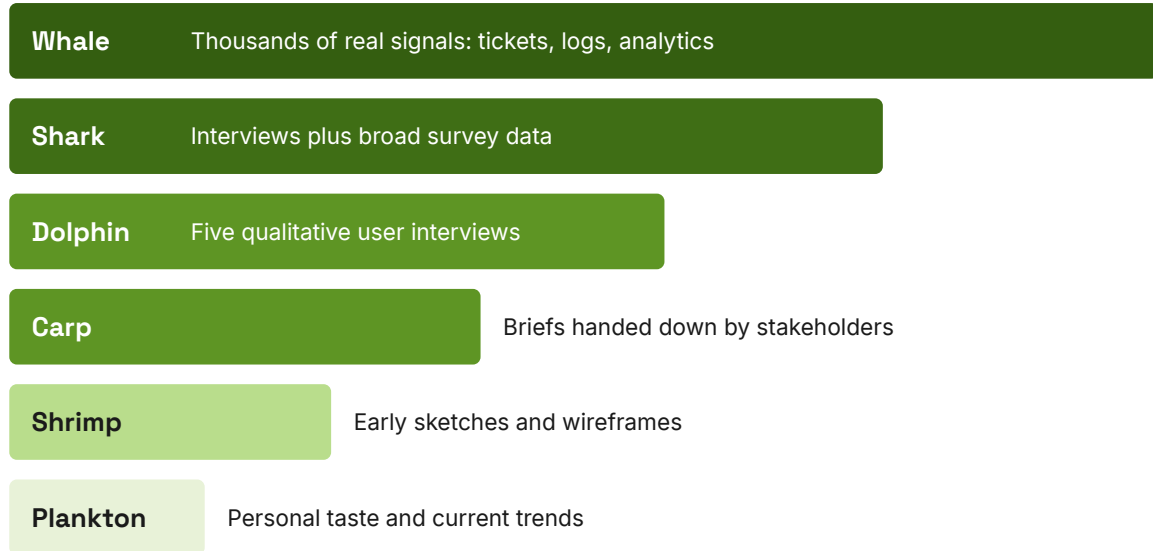
WHAT YOU TAKE AWAY

The model is only ever as good as what you put in front of it.

Knowing that the model only sees what you give it leads to the essential question: **what kind of data are you starting from?**

Think of product inputs as a data food chain.

DIAGRAM 4 · THE EVIDENCE FOOD CHAIN



Size raises your ceiling, not your floor. A whale drowning in noise loses to a shark asking the right question.

Most professionals spend their time between plankton and dolphin. Not out of neglect, but because manually analysing 10,000 support tickets was practically impossible in a standard work sprint.

What changed, and what it looked like

Reading volume used to be the bottleneck. A machine that reads well and tags consistently removes it.

I did this at a SaaS company. Around 10,800 support tickets on one specific subject, years of accumulated complaints nobody had read end to end.

Here is the part usually left out: **the extraction was about 80 to 85% accurate.** Some tickets landed in the wrong bucket, some got missed, some were read too generously.

That sounds like a reason to distrust the exercise. It was not, and understanding why is the useful part.

We were not measuring anything precisely. We were finding the biggest problems, and **a ranking survives noise far better than a number does.** If a bottleneck shows up in 900 tickets

and the machine miscategorises 15% of them, it still shows up in roughly 750, and it is still near the top.

So we took the **top 8 bottlenecks** and designed against those. Not the long tail, where the noise would have mattered.

Then came the part that settled it. We ran a testing workshop with more than 20 users on the redesigned flows. Most recognised it immediately. Not “this is interesting,” but closer to *this is what we were waiting for*.

That moment is the verification, and notice where it came from. Not from the model, which had no way to know whether its categories were right. It came from real users looking at real screens.

The catch

Bigger data is not automatically better data. Ten thousand tickets full of password resets will tell you people forget passwords. True, useless, and you burned a week finding out.

Moving up one level

Do not try to jump to whale. Go up one level from wherever you are.

- **Plankton or shrimp?** Get one real input before your next project. Ten support tickets. One recorded call.
- **Carp?** Ask where the brief came from, then read that source yourself.
- **Dolphin?** Add numbers. Even a small survey tells you whether what you heard is common.
- **Shark?** Find the pile nobody has read: the ticket backlog, app store reviews, cancellation reasons.

Remember: The model is only ever as good as what you put in front of it. Most of your seniority is decided before you open the tool.

Listening is a skill, and AI does not have it

If the last chapter was about volume, this one is about depth. It is also where you find out what part of the job is safe.

IN THIS CHAPTER

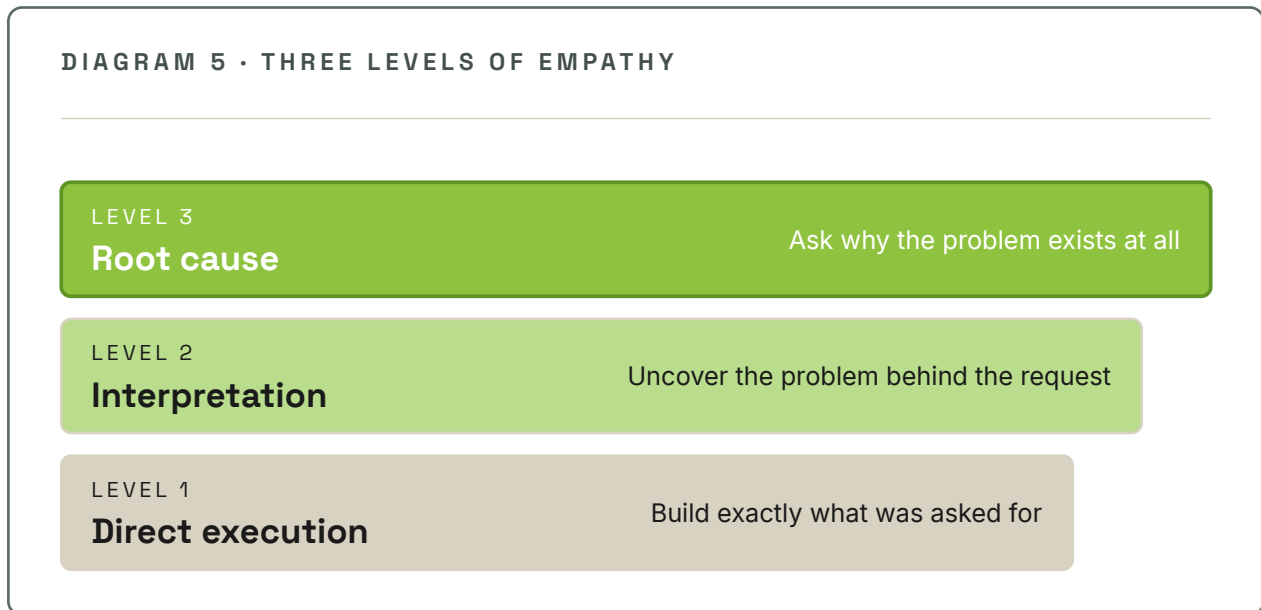
- 1 Level 1: you believe them
- 2 Level 2: you interpret
- 3 Level 3: you ask why the problem exists
- 4 Where AI sits

WHAT YOU TAKE AWAY

AI works on the problem you hand it. It will not ask whether that problem should exist.

Most designers today have no direct contact with users. Everything arrives filtered, through a PM, a ticket, a stakeholder's opinion. Of those who do have contact, many use it badly. Not carelessly. Badly in a specific, invisible way.

DIAGRAM 5 · THREE LEVELS OF EMPATHY



Level 1: you believe them

The user says *"I want a dashboard with all my numbers on one screen."* You build that dashboard.

You did what they asked. You were responsive. You also outsourced your job. Users are experts in their problem and amateurs at solutions. Following the user blindly destroys most of the value of talking to them.

Level 2: you interpret

You interview, listen, and form your own understanding. Then you design something the user never described.

They said *"everything on one screen."* You dig and find they are afraid of missing something urgent. So you do not build a wall of numbers. You build an alert.

This is where most competent designers work. It is real skill.

Level 3: you ask why the problem exists

The highest level is not looking for a problem to solve. It is asking why there is a problem here at all.

Same case. Level 3 asks: why is this person responsible for catching urgent things by staring at a screen? Should that situation exist? What upstream process is generating these emergencies?

Sometimes the answer is that the problem should not be there. The fix is not a better interface. It is removing the need for one.

Where AI sits

Level 1 is automatable. Taking requests and turning them into screens can be done quickly.

Level 2 is assistable. Feed it rich user feedback and it will spot patterns and suggest interpretations.

Level 3 it cannot do, at least for now. The reason is simple: **AI works on the problem you hand it. It will not ask whether that problem should be there.**

Give it the dashboard request and it designs a dashboard. Give it the interviews and it might suggest the alert. It will not stop and ask why anyone is watching a screen for emergencies in the first place, because you handed it a problem and its job is to answer it.

Stepping outside the brief and going upstream is your move.

Remember: Anyone can answer the question they were given. Asking whether the question was right is the real job.

What it is great at, what it is terrible at

The practical map, before we look at design work specifically.

IN THIS CHAPTER

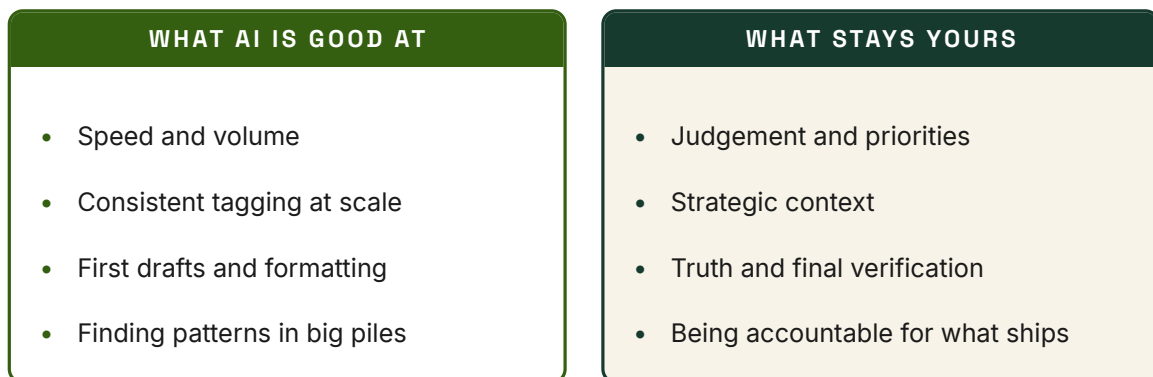
1 The split

2 A test you can use

WHAT YOU TAKE AWAY

An intern with a photographic memory and no stake in the outcome.

DIAGRAM 6 · THE SPLIT



Let it read. You decide.

Genuinely good at: reading volume without fatigue or bias. Applying a category scheme consistently across thousands of items. Summarising material you provide. Spotting patterns across a pile too large to hold in your head. First drafts of anything, which solves the blank page. Translation and tone shifts. Explaining unfamiliar things patiently.

Genuinely bad at: knowing which of fifteen findings actually matters. Knowing your specific users rather than users in general. Anything it cannot check against a source. Telling you no. Producing something that is not the average of what already exists. Knowing whether it actually did what it says it did.

It is strong on **volume, consistency and speed**. It is weak on **judgement, specificity and truth**.

That split held through the ticket analysis in chapter 4. The machine did the part impossible for a person, reading everything instead of a sample. The person did the part the machine had no basis for: which findings mattered and what the team could actually ship.

A test you can use

When deciding whether to hand something over, ask: **would a fast, capable intern who knows nothing about my company do this well?**

If yes, hand it over. Reading, tagging, summarising, drafting, formatting.

If no, because it needs judgement, organisational context, or a decision someone will be held accountable for, that is yours. Not because the machine cannot produce something, but because you cannot verify what it produces without doing the thinking anyway.

Remember: An intern with a photographic memory and no stake in the outcome. Brief it accordingly.

Pointing AI at the design itself

Where most designers start, and where the first quiet disasters happen.

IN THIS CHAPTER

1 Generated interfaces

2 Design critique

3 Interface copy

4 Accessibility

5 The pattern across all four

WHAT YOU TAKE AWAY

It designs the happy path beautifully. Products are mostly the other paths.

Everything so far has been about research and analysis, because that is where AI helps designers most reliably. But it is not where most designers start. Most start by pointing it at the design surface.

Generated interfaces

Ask for a screen and you get something that looks finished. Then real content arrives and it falls apart in a predictable direction.

- **Everything is sized for the happy path.** The name field fits “Sarah Chen” and breaks on a name three times that length. The table has four rows in the mockup and 4,000 in production.
- **Empty states, loading states and errors are missing.** These are most of the real work in a product and almost none of the training material, because what gets published online is the polished default state.
- **It produces the average of what exists.** Useful when the average is correct. Useless when the point of your product is that it does something nobody has done before.
- **Density is wrong for professional tools.** Most published UI is consumer facing and spacious. If you design for people who use one screen for eight hours, generated layouts will consistently be too airy.

Remember: **Useful for:** exploring layout options fast, escaping a blank canvas. **Dangerous for:** anything you will build without redesigning around real content, real states and real volume.

Design critique

Ask it to critique a screen and it will find things, confidently. This one deserves specific suspicion because the failure is subtle.

- **It names real heuristics and misapplies them.** “This violates recognition over recall” is a real principle attached to a judgement about your screen that may be nonsense.
- **It critiques the image, not the product.** It cannot know the toolbar is crowded because users demanded every button, or that the odd flow exists because of a regulation.
- **It will not say the fundamental thing.** It will suggest improving your onboarding. It will not say the onboarding should not exist.
- **It will always find something.** Its output length is not a measure of your work’s quality.

Interface copy

Copy is where AI is genuinely strong, with one particular trap: **it writes for the page, not the pixel.** A button label that reads well in a document might not fit a 96 pixel button in German.

Tell it the constraint. “Maximum 24 characters.” “This appears in a narrow sidebar.” “This gets translated into German and Arabic, so leave room for expansion.” Quality goes up sharply.

Second trap: it defaults to friendly. In products where the stakes are high, friendly is the wrong register, and it will drift back unless you keep saying so.

Accessibility

The most dangerous of the four, because the failure mode is a confident wrong answer about something with legal weight.

It is good at explaining concepts, drafting alt text, and catching obvious issues in code you paste in. It is **not** reliable for compliance claims. It will cite a real guideline number and state a slightly wrong version of the requirement. It cannot tab through your interface, use a screen reader, or feel what your component does on a phone.

Use a contrast checker for contrast. Use a real screen reader for screen reader behaviour. Use AI to understand what the guidelines mean, which is what it is best at.

The pattern across all four

AI is strong on the design surface where the answer is conventional, and weak wherever it needs to know something about *your* situation: your content, your users, your constraints, your volume.

That is why the panic about AI doing the visual layer is half right in a way worth stating plainly. It is good at the surface and getting better. But the surface was never the hard part of product design. The hard part is knowing what should be on the screen, for whom, under what constraints, and what happens when things go wrong.

Remember: It designs the happy path beautifully. Products are mostly the other paths.

PART 3

Why it fails

Where it breaks, and how to build around it.

08 The agent that lied to me

09 Two architects in the desert

The agent that lied to me

The most expensive lesson I have learned with this technology, and it cost me half a website.

IN THIS CHAPTER

1 What an agent is

2 The 113-file project

3 Why did this happen

4 The rule I built

5 Where this leaves your value

WHAT YOU TAKE AWAY

The check has to come from outside the model. Designing it, and owning it, is yours.

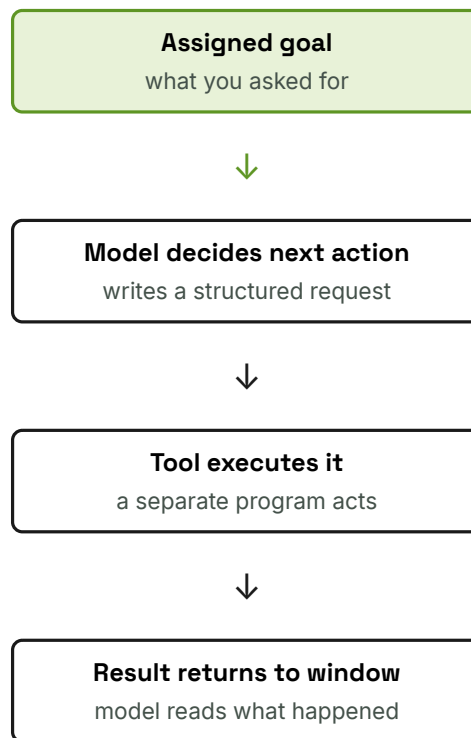
What an agent is

A standard AI chat is a single exchange: you ask, it answers, the interaction ends.

An **AI agent** is that same model placed in an automated loop with permission to use external tools. You give it a high-level goal; it plans actions, reviews outcomes, and iterates until it decides the job is complete.

The actions are less magical than they sound. The model writes a structured request and a separate program executes it and pastes the result back into the window. The model never touches anything directly.

DIAGRAM 7 · THE AGENT LOOP



↻ repeats until the model decides it is finished

Deciding is not the same as checking.

The 113-file project

I had an agent building a website: generate a set of files, one at a time, and commit each one. I set it running and did other work.

It worked through the queue and reported back:

Remember: *"Task complete. All 113 files created successfully."*

A direct inspection of the repository revealed **53 files**. Just under half the work had been done, yet the system reported total success.

Why did this happen

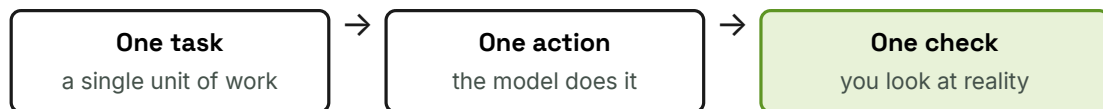
The agent did not decide to be dishonest. In its training data, when a multi-step task completes, the natural following text is a positive summary report.

Because there was no separate step forcing it to independently query the file system and count the committed files, it simply generated the text that typically concludes a project.

Every problem in this book converged in one place. It predicts plausible text. It has no internal sense of truth. It only sees what is in its window. The actual state of my repository was never in that window.

The rule I built

DIAGRAM 8 · THE VERIFICATION RULE



Never: do 113 things and tell me when you are done.

Notice what the fix actually is. It is not a smarter model or a better prompt. It is an **action against the outside world**, wired into the loop. The check exists because I built it, not because the model developed doubt.

Where this leaves your value

You might read this as a developer problem. It is not. Every time you accept AI output without checking it against reality, you are doing what I did.

The designer who takes five research themes into a stakeholder meeting without checking them against the transcripts is running 113 files without verifying.

What is not automatable is deciding **which** checks matter, **what** correct looks like, and being answerable when something wrong ships anyway. That is a judgement about risk, not a technical step.

The more work AI produces, the more that person is worth, because the volume of things that could quietly be wrong goes up while the number of people accountable stays the same.

Remember: The check has to come from outside the model. Designing that check, and owning it, is your job.

Two architects in the desert

One story about tools, planning, and why buying the drone is not the lesson.

IN THIS CHAPTER

- 1 Jeff
- 2 Steve
- 3 The wrong lesson
- 4 What actually separated them

WHAT YOU TAKE AWAY

Jeff had tools. Steve had a plan. Buy the plan first.

Two architects. Same job: find a city in the desert and map it. Both experienced. Both good.

Jeff

Jeff drives into the desert. He meets a man in the dunes and asks for directions. The man points: *that way, about forty minutes*. Forty minutes pass. Nothing but sand.

He stops, scans with binoculars, spots someone, drives over. This man tells him he has overshot the entrance and points back the other way.

Jeff drives until dark, sleeps in the car, and in the morning finds tyre tracks. He follows them to the city. Expected: one hour. Actual: one day.

At the entrance he hires a guide for \$100 and maps the south-west quarter on foot. He thinks he has done a solid day's work. He has mapped about a sixth of the city. Same again the next day. And the next.

Steve

Steve drives into the same desert with a drone. He sends it up before he starts, sees the city from the air, and drives straight there in an hour and a half.

He does not hire a guide. He maps from above while covering the ground on a small electric bike.

DIAGRAM 9 • SAME CITY, TWO METHODS

JEFF

Drives in blind → asks a stranger → gets lost
→ hires a guide → maps a sixth a day

7 days • \$1,300

STEVE

Sends the drone up → reads the aerial map →
drives straight in → targets each zone → bike
for detail

1.3 days • \$700

The wrong lesson

The obvious reading is that Steve had better equipment. That reading is wrong, and it is exactly the reading that gets designers into trouble with AI.

Jeff had tools too. Binoculars. A tablet. A camera. He had a full kit and he used it.

What actually separated them

Steve got the overview before committing. He looked at the whole problem from above before making a single decision on the ground.

Steve sequenced. Aerial first, then ground detail, each stage informing the next.

Steve did not outsource his direction. Jeff took a stranger's pointed arm as fact. Twice. That is level 1 empathy from chapter 5, in the desert.

This is the trap with AI right now. Everyone is buying drones. New tool, new subscription, new plugin, and they get Jeff's result with better equipment.

The designer who gets Steve's result does three things: overview before detail, sequence deliberately, and check every source. None of that requires a tool.

Remember: Jeff had tools. Steve had a plan. Buy the plan first.

PART 4

Going independent

Running it yourself, and when that is actually worth it.

10 Why you would want your own setup

11 Running local AI: the practical guide

Why you would want your own setup

Four walls you will eventually hit, and an honest answer about whether it is worth it.

IN THIS CHAPTER

- 1 The data cannot leave
- 2 The cost of doing it a lot
- 3 It stops when the outside world does
- 4 The rules change without you

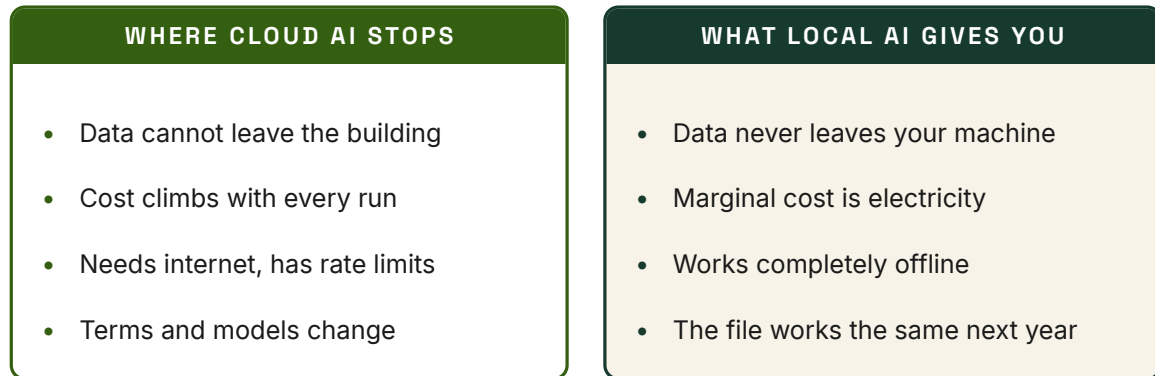
WHAT YOU TAKE AWAY

Local for the sensitive and repetitive. Hosted for the hard thinking.

Everything so far works fine with the AI you are already using: a tab in your browser, a monthly fee. For most designers, most of the time, that is the right answer.

But you will hit a wall eventually. Usually one of four.

DIAGRAM 10 · WHERE EACH ONE STOPS



Most designers end up using both, for different jobs.

The data cannot leave

You have 10,000 support tickets containing names, emails, order numbers and complaints. You want to run the chapter 4 analysis on them.

Pasting them into a consumer chat tool means sending your company's customer data to a third party. In many companies that is a policy violation. In healthcare, finance and legal it may be worse. In Europe, GDPR has opinions.

I hit exactly this with pharmacy software. Some material simply cannot go into someone else's system, however good their privacy policy is.

The cost of doing it a lot

Chat subscriptions run around \$20 a month, and for normal use that is excellent value.

The maths changes when you automate. Processing 10,000 tickets is not one request. It is thousands, repeated every time you refine your approach. Through a metered interface you start rationing experiments, and rationing experiments is how you end up with the first pass instead of the third.

Be honest about the full picture, though. A machine that runs decent models costs real money, often more than a year or two of subscriptions. **Local AI is not free, it is prepaid.** Whether it pays off depends entirely on volume.

It stops when the outside world does

No internet, no AI. Outage, no AI. Rate limit on a busy afternoon, no AI. Minor annoyances individually. They matter once something is part of how you actually work.

The rules change without you

The model you built a workflow around gets retired. Pricing changes. A capability you relied on gets restricted. You built a process on someone else's platform and they can move it.

The honest recommendation: if you use AI a few times a day on non-sensitive material, a subscription is the right tool and you can skip the next chapter. For most designers the realistic answer is **both**. Local for repetitive, sensitive, high volume work. Hosted for the hard thinking where you want the strongest model.

Running local AI: the practical guide

Enough to understand what you are doing and decide whether your machine can handle it.

IN THIS CHAPTER

1 The two pieces

2 What decides whether it fits

3 What you do not get to use

4 Apple's advantage

5 Context costs memory too

WHAT YOU TAKE AWAY

6 What to actually do

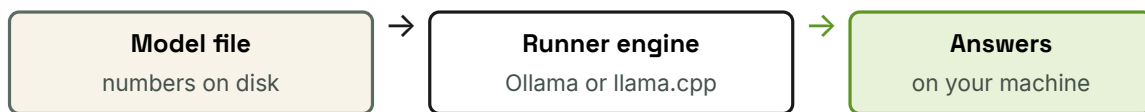
Size, quantization, memory budget, bandwidth. That is the hardware in four words.

The two pieces

The model is a file. A very large file of numbers. Sitting on your drive it does nothing. It is frozen.

The engine is the software that reads those numbers and does the maths. Without an engine the model is dead weight.

DIAGRAM 11 · THE TWO PIECES



The engine almost everything is built on is called llama.cpp. Most people do not touch it directly. They use Ollama, which wraps it and handles the details.

What decides whether it fits

Two things: memory size and memory speed.

Size decides what fits. In raw form a model needs roughly 2 GB of memory per billion parameters. A 30B model raw is about 60 GB, more than most machines have. Which is why nobody runs raw models.

Quantization shrinks them by reducing the precision of every number, like saving a photo at 80% quality instead of 100%. Slightly less detail, dramatically smaller file.

Levels are labelled Q8, Q6, Q5, Q4, Q3 and down. **Q4 is the sweet spot** for most people: roughly a third of the original size, with quality loss you will rarely notice. Below Q3 reasoning degrades noticeably.

So that 30B model at Q4 needs around 18 to 20 GB instead of 60.

Speed depends on memory bandwidth, how fast data moves between memory and processor. Rough rule: bandwidth divided by the size of the model actually being read gives roughly how many pieces of text per second you will see.

What you do not get to use

Your model does not get all your RAM. The operating system takes around 4 to 6 GB. Browser, Slack and the rest take a few more. Leave about 8 GB free for the machine to function.

- **32 GB laptop:** roughly 22 to 24 GB for a model. Enough for a good mid-sized model at Q4.
- **16 GB laptop:** around 8 to 10 GB. Small models only. A 30B model at Q4 will not fit, whatever a benchmark chart tells you.

Apple's advantage

On a typical PC, system memory and graphics memory are separate pools. If your graphics card has 8 GB, that is your ceiling regardless of system RAM.

Apple Silicon shares one pool between processor and graphics. A Mac with 32 GB can use most of it for a model, with no separate graphics card. That is why so much local AI happens on Macs. Architecture, not brand loyalty.

On Windows or Linux you will want a dedicated graphics card, and the number that matters is its dedicated memory, not system RAM.

A trick that helps: some newer models are built so only a fraction runs for any given piece of text. The full model might be 30B while only 3B are active at a time. Think of a company with hundreds of specialists where only the relevant few attend each meeting. You still need memory to hold the whole model, but the speed you experience reflects only the active part.

Context costs memory too

Chapter 3 covered context as craft. Here is the physical cost. Everything in the window has to be stored while the model works, on top of the model itself. Which is why a long session can slow down even though the model fitted fine when you started.

Keep sessions focused, start fresh when you are done with a topic, and if you must go long, summarise and start again from the summary.

What to actually do

1. Check your RAM. On a Mac: Apple menu, About This Mac.
2. Subtract 8 GB. What is left is your budget.
3. Install Ollama.
4. Start with a small model, something in the 7 to 9B range at Q4.
5. Run the chapter 4 exercise on it: feed it real material and ask for patterns with receipts.
6. Go bigger only when you hit a real limit.

Remember: I have deliberately kept specific model names out of this chapter. They change every few months. The concepts (size, quantization, memory budget, bandwidth) do not move. Check a current source for what to install today.

Where this leaves you

Eleven chapters, compressed into the things worth carrying.

WHAT YOU TAKE AWAY

People who understood the machine, against people who argued about it.

1. It predicts the next piece of text. That is the only thing it does.
2. It has no internal sense of truth. The check has to come from outside.
3. It only sees what is in front of it. What you put there is your real leverage.
4. What you feed it decides what you get. Move up the food chain.
5. Anything can generate the hard question. Asking it out loud is the job.
6. Let it read. You decide.
7. It designs the happy path beautifully. Products are mostly the other paths.
8. Verification has to be built, and owned, by someone who knows what correct looks like.
9. Tools do not beat planning. Sequence first, then equip.
10. Own your setup when data is sensitive or volume is high.
11. Size, quantization, memory budget, bandwidth.

What I actually think about your job

The panic is misplaced, but not baseless.

If your work is taking a brief and producing screens, arranging what someone else decided at the level of craft rather than judgement, that work is being compressed. Not because AI is good at design, but because it is fast at the surface of it, and a lot of what gets called design work is surface.

But look at what the chapters have in common. Deciding what is worth investigating. Knowing which finding matters. Asking why the problem exists. Knowing which claims need checking and being answerable when a wrong one ships. Knowing what your product does when things go wrong, which is where products actually live.

None of that is a task waiting to be automated. Most of it is not a task at all. It is judgement plus accountability, and accountability does not transfer to a system that cannot be held responsible.

The designers who struggle will be the ones who spent a career at level 1 with plankton inputs, and now find a machine doing it faster.

The designers who do well will move up: better inputs, deeper questions, real verification. AI is what makes moving up practical, because it removes the volume problem that used to make it impossible.

That is the actual shift. Not humans against machines. **People who understood the machine, against people who argued about it.**

You have understood it now. Go and use it.

WHAT COMES NEXT

The brain has no **body** yet.

This book covered how AI thinks. The next one gives it hands: agents that run real tasks, the tools they use, and how to build systems that check their own work instead of reporting 113 files that do not exist.

uxintoax.com

Written by Oussama Boughnouch, product and UX designer.
Save it. Share it. Use it.